深度学习计算

> 注：这一讲主要是和代码实现相关，本笔记侧重于理论知识，故简略带过

层和块

- **层**：就是最基础的神经网络的组成架构。例如一个最简单的多层感知机（下图左），包含一个输入层，一个隐藏层，一个输出层
- **块**：就是层的打包，也可以是某些块的打包（下图中、右）

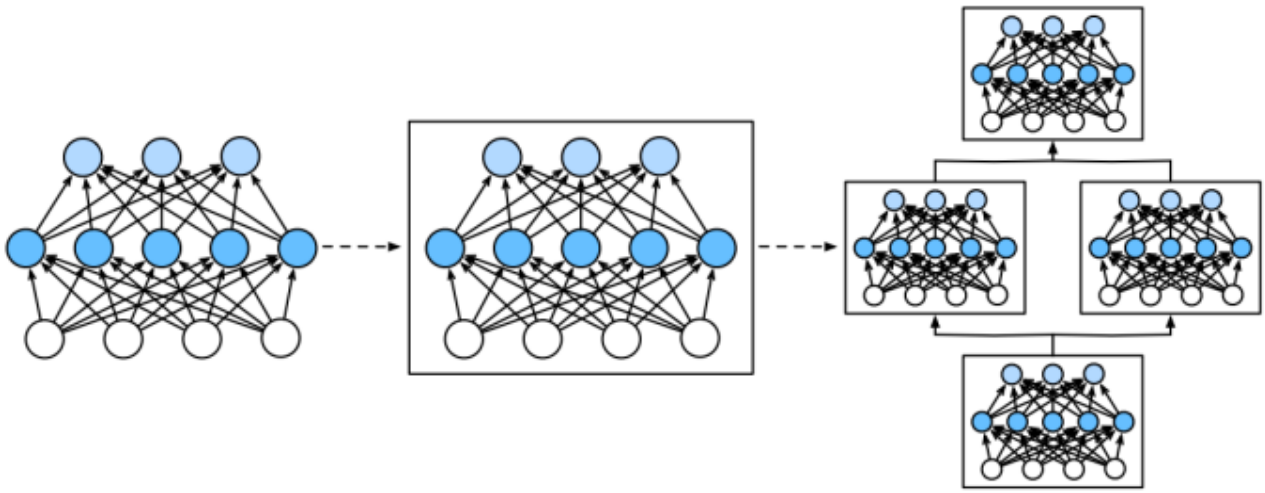

图 1 多个层被组成块，形成更大的模型

要点：

- 块可以包含代码
- 块负责大量的内部处理，包括参数初始化和反向传播
- 层和块的顺序连接由 `Sequential` 块处理

我们可以访问、初始化和绑定模型参数，可以使用自定义初始化方法。

延后初始化

有时我们**不把网络中每一层的输入维度写死，只告诉输出维度**，例如：

```
net = nn.Sequential(
    nn.LazyLinear(256),
    nn.ReLU(),
    nn.LazyLinear(10)
)
```

其实可以理解成：

- 第 1 层：`Linear(?, 256)`
- 第 2 层：`Linear(?, 10)`

假设第一次输入 `X = torch.rand(4, 20)`：

- 发现输入的最后一维是 20，则第一层补全为 `Linear(20, 256)`，X 经过后形状变为 $(4, 256)$
- 经过 `nn.ReLU()` 不改变形状
- 输入第二层，发现此时输入的最后一维是 256，则第二层补全为 `Linear(256, 10)`

总而言之：延后初始化使框架能够自动推断参数形状，使修改模型架构变得容易，避免了一些常见的错误。我们可以通过模型传递数据，使框架最终初始化参数。

自定义层

我们可以通过基本层类设计自定义层。这允许我们定义灵活的新层，其行为与深度学习框架中的任何现有层不同。

在自定义层定义完成后，我们就可以在任意环境和网络架构中调用该自定义层。

层可以有局部参数，这些参数可以通过内置函数创建。

读写文件

加载和保存张量 / 模型参数（略）。

本章涉及的代码操作见于 `Chapter5demo.py`